
AQI Predictor

*Real-time 24-Hour Air Quality Index Forecasting
for 5 Major Indian Cities*

End-to-End MLOps Project Report

ML Model:	LightGBM (MultiOutputRegressor, 24-step horizon)
Backend:	FastAPI 0.115 + MLflow 3.11 Model Server
Frontend:	Streamlit + Plotly
Orchestration:	Apache Airflow 2.x (hourly DAG)
Monitoring:	Prometheus + Grafana
Versioning:	Git + DVC (9 tags, 11-stage pipeline)
Containerised:	Docker Compose (5 services + Airflow stack)
Data Sources:	CPCB / Kaggle, OpenAQ v3, Open-Meteo Archive
Cities:	Chennai, Delhi, Mumbai, Kolkata, Ahmedabad
Report Date:	April 2026
Version:	1.0

This report covers the design, data pipeline, model training, deployment, monitoring, and testing of the AQI Predictor MLOps system.

Contents

1	Abstract	6
2	Problem Statement and Objectives	6
3	System Architecture	7
3.1	High-Level Architecture	7
3.2	Docker Compose Services	8
3.3	Design Principles	8
4	Data Sources and Collection	9
4.1	Training Data — CPCB (Kaggle)	9
4.2	Weather Data — Open-Meteo Archive API	10
4.3	Live Production Data — OpenAQ v3	10
5	Exploratory Data Analysis (EDA)	11
5.1	Key Findings	11
5.2	Selected Visualisations	12
6	Data Pipeline and Preprocessing	12
6.1	DVC Pipeline Overview	12
6.2	Version History (Git Tags)	15
6.3	Feature Engineering	15
7	Model Development and Experiment Tracking	16
7.1	Modelling Strategy	16
7.2	Models Attempted	16
7.3	Hyperparameters — Best Model (lgbm-exp4)	17
7.4	MLflow Experiment Tracking	17
7.5	Experiment Results	18
7.6	Model Deployment Bundle	18
8	Backend — FastAPI Application	19
8.1	Application Structure	19
8.2	Startup Lifecycle	20
8.3	API Endpoints	20
8.4	/predict Endpoint — Step-by-Step Flow	21
8.5	/predict Response Schema	21

8.6	Error Handling	22
8.7	Prometheus Metrics	23
9	Frontend — Streamlit Dashboard	23
9.1	Overview	23
9.2	Dashboard Layout	24
9.3	User Manual Page	25
10	Data Ingestion Pipeline — Apache Airflow	25
10.1	DAG Overview	25
10.2	Ingestion DAG Tasks	26
11	Drift Detection	27
11.1	Rolling Window Strategy	27
11.2	Statistical Signals	27
11.3	Severity Classification	28
12	Monitoring — Prometheus and Grafana	28
12.1	Architecture	28
12.2	Alert Rules	29
13	Database Design	30
14	Testing	30
14.1	Test Strategy	30
14.2	Test Execution and Results	31
14.3	Acceptance Criteria	31
15	MLOps Implementation Summary	32
16	Known Limitations and Future Work	32
16.1	Current Limitations	32
16.2	Planned Improvements (Phase 2)	33
17	Project Structure	33
18	Conclusion	35
A	Appendix A — Environment Variables	36
B	Appendix B — params.yaml (Full)	36
C	Appendix C — Backend Requirements	38

D Appendix D — Quick-Start Commands	39
--	-----------

List of Figures

1	High-Level System Architecture — five independent layers.	7
2	Key EDA findings from CPCB training data (2015–2020).	12
3	DVC pipeline DAG — 11 stages from raw data to model evaluation. . . .	13
4	MLflow Runs UI — all 8 experiments sorted by <code>val_mae</code> . The best run (<code>lgbm_20260412_010953</code>) is registered as <code>AQI_LightGBM_v2</code> in the Produc- tion stage.	18
5	Complete step-by-step flow of the <code>/predict</code> endpoint.	21
6	Streamlit dashboard showing a 24-hour AQI forecast for a Chennai station.	24
7	Apache Airflow DAG graph view: <code>bootstrap</code> → <code>ingest</code> → <code>cleanup</code> → <code>drift</code> .	26
8	Grafana monitoring dashboard — prediction latency, request rate, model readiness, data freshness, and per-feature drift PSI.	29

List of Tables

1	Project success metrics and achieved results	7
2	Docker Compose services — ports and roles	8
3	CPCB dataset files	9
4	Pollutant columns in training data	10
5	DVC pipeline stages	14
6	9 Git tags — data and model version history	15
7	Feature groups generated by <code>preprocess.py</code>	15
8	Models evaluated during the project	16
9	Final LightGBM hyperparameters (<code>params.yaml</code>)	17
10	All experiment results from MLflow (sorted by <code>val_mae</code>)	18
11	FastAPI endpoint specification	21
12	HTTP error codes returned by <code>/predict</code>	22
13	Custom Prometheus metrics instrumented in <code>utils/metrics.py</code>	23
14	User manual page tabs	25
15	Airflow DAGs — schedules and purposes	25
16	Drift detection windows	27
17	Drift severity levels and PSI thresholds	28
18	Prometheus alerting rules (<code>monitoring/alert_rules.yaml</code>)	29
19	SQLite database tables and retention policy	30

20	Test suite breakdown by module	31
21	Test execution results	31
22	MLOps checklist against evaluation criteria	32
23	All configurable environment variables (<code>.env</code>)	36

1 Abstract

Air pollution is a critical public health crisis in India. This project, **AQI Predictor**, delivers a full-stack production-grade MLOps system that ingests live pollutant and weather data, trains an ensemble ML model, and serves real-time 24-hour AQI forecasts through a REST API and a Streamlit dashboard.

The best model — a **LightGBM MultiOutputRegressor** with a 24-step multi-output design — achieves a validation MAE of **26.25** and **$R^2 = 0.80$** , outperforming all 7 other experiment runs tracked in MLflow. The system is fully containerised with Docker Compose, orchestrated via Apache Airflow, and monitored with Prometheus and Grafana. Data versioning is managed with DVC (9 Git tags, 11-stage pipeline), and the test suite covers **47 automated pytest cases** with a 100% pass rate.

2 Problem Statement and Objectives

India's CPCB operates hundreds of air-quality monitoring stations across major cities, but real-time 24-hour AQI forecasts are not publicly accessible in an actionable format. This project addresses that gap by:

1. Ingesting live hourly sensor data from OpenAQ v3 CPCB stations.
2. Enriching readings with weather variables from Open-Meteo.
3. Predicting the next 24 AQI values ($t + 1$ through $t + 24$) per station.
4. Serving forecasts via a clean REST API and a Streamlit UI with health advisories.
5. Monitoring for data drift and model degradation continuously.

Success Metrics

Table 1: Project success metrics and achieved results

Type	Metric	Target	Achieved
ML	Validation MAE (overall)	< 30	26.25
ML	R ² on held-out test set	> 0.75	0.80
System	API P95 latency	< 10s	
System	Data freshness	< 3h old	
Drift	Retrain trigger	> 40% drift	
Testing	Test pass rate	100% (47/47)	47/47

3 System Architecture

3.1 High-Level Architecture

The system is structured as five independent layers connected via REST APIs and a shared SQLite database. No layer has a direct in-process dependency on any other.

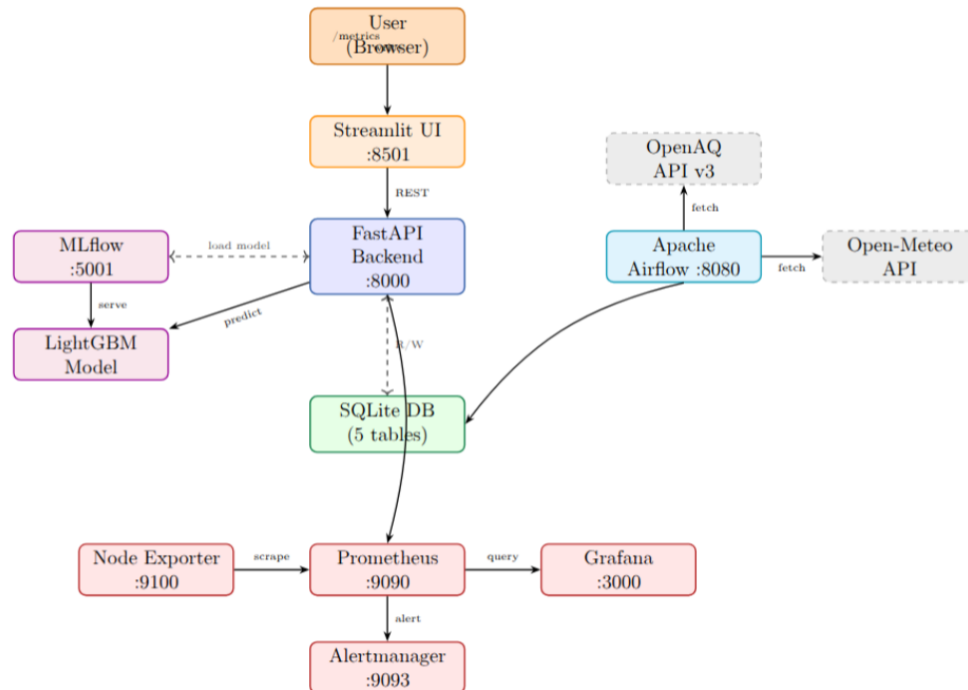


Figure 1: High-Level System Architecture — five independent layers.

3.2 Docker Compose Services

Table 2: Docker Compose services — ports and roles

Service	Port	Technology	Role
backend	8000	FastAPI + Uvicorn	REST API, prediction serving, Prometheus metrics
frontend	8501	Streamlit + Plotly	User-facing forecast dashboard
mlflow	5001	MLflow models serve	Model inference via HTTP JSON (/invocations)
prometheus	9090	Prometheus	Metrics scraping from /metrics endpoint
grafana	3000	Grafana	Real-time dashboards and alert routing
<i>Airflow stack — managed separately via <code>airflow/docker-compose.yml</code></i>			
airflow-webserver	8080	Apache Airflow 2.x	DAG UI, trigger management
airflow-scheduler	—	Apache Airflow 2.x	DAG scheduling and task execution
airflow-postgres	5432	PostgreSQL	Airflow metadata database

Airflow Deployment Note

Apache Airflow is **not included in the main `docker-compose.yml`**. It is deployed separately using the official Airflow Docker Compose stack located at `airflow/docker-compose.yml`. The Airflow containers share the project's SQLite database by mounting `backend/database/` as a volume, enabling DAG tasks to read and write `live_aqi_buffer.db` directly. To start Airflow independently:

```
cd airflow
docker-compose up -d
```

3.3 Design Principles

- **Loose coupling:** Streamlit communicates with FastAPI only via REST. No shared in-process state between any two services.
- **Stateless functional services:** Backend logic is split into 11 domain service modules — `db_service`, `model_service`, `drift_detection`, `openaq_service`, `weather_service`,

`advisory_service`, `bootstrap_service`, `ingestion_service`, `live_pipeline_service`, `feature_service`, `retention_service`.

- **Reproducibility:** Every experiment is reproducible via a Git tag combined with its MLflow run ID.
- **Fail-fast startup:** The backend validates all 6 deployment bundle artifacts at startup and raises `FileNotFoundError` before accepting any traffic if any file is missing.

4 Data Sources and Collection

4.1 Training Data — CPCB (Kaggle)

Training data was sourced from the **CPCB Air Quality Dataset** on Kaggle, covering 2015–2020 across 110 monitoring stations in 26 Indian cities.

Table 3: CPCB dataset files

File	Key Columns	Description
<code>station_hour.csv</code>	StationId, Datetime, 12 pollutants, AQI	Hourly pollutant readings (2,319,535 rows)
<code>stations.csv</code>	StationId, City, State, Lat, Lon, Status	Station metadata (230 records)

Table 4: Pollutant columns in training data

Column	Pollutant	Unit	Null %
PM2.5	Fine particulate matter	$\mu g/m^3$	16.3%
PM10	Coarse particulate matter	$\mu g/m^3$	36.6%
NO	Nitric oxide	$\mu g/m^3$	12.3%
NO2	Nitrogen dioxide	$\mu g/m^3$	11.2%
NOx	Nitrogen oxides	$\mu g/m^3$	9.5%
NH3	Ammonia	$\mu g/m^3$	41.7%
CO	Carbon monoxide	mg/m^3	9.9%
SO2	Sulphur dioxide	$\mu g/m^3$	20.4%
O3	Ozone	$\mu g/m^3$	19.7%
Benzene	Benzene	$\mu g/m^3$	25.5%
Toluene	Toluene	$\mu g/m^3$	33.3%
Xylene	Xylene	$\mu g/m^3$	77.8%
AQI	Target variable	Index (0–500)	14.4%

4.2 Weather Data — Open-Meteo Archive API

Weather history was fetched per-city using the Open-Meteo Archive API (`archive-api.open-meteo.com`) with 7 variables: `temperature_2m`, `relative_humidity_2m`, `precipitation`, `pressure_msl`, `cloud_cover`, `wind_speed_10m`, `wind_direction_10m`.

Fetching uses exponential backoff (30s, 60s, 90s delays) and a per-city JSON cache to support resumable runs after rate-limit interruptions.

4.3 Live Production Data — OpenAQ v3

In production, the `openaq_service.py` module polls individual sensors from the OpenAQ v3 API (`api.openaq.org/v3`). All units are normalised to SI standards:

- $ppb \rightarrow \mu g/m^3$ via molecular weight conversion
- $ppm \rightarrow mg/m^3$ (for CO)

Rate limiting is handled with a `Retry-After` header parser and 429-aware exponential backoff.

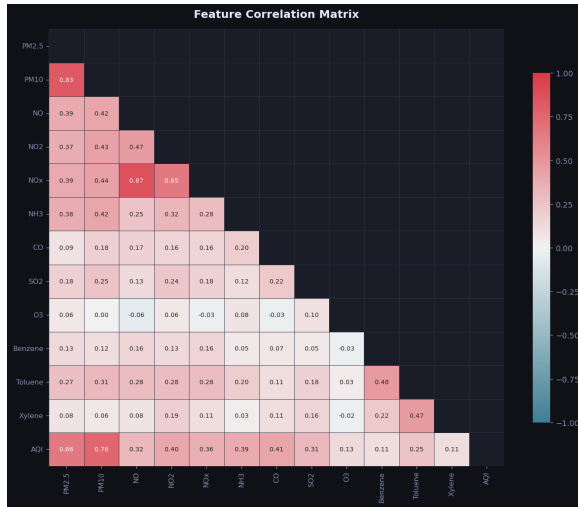
5 Exploratory Data Analysis (EDA)

EDA was performed using a dedicated `eda.py` script that produces 15 visualisations and saves `drift_baseline/baseline_stats.json` (per-feature mean, std, p25, p50, p75, p95, skewness, kurtosis) for downstream drift detection.

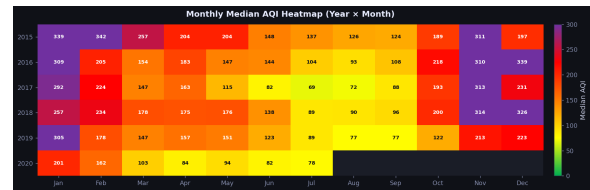
5.1 Key Findings

1. **Dataset scale:** The CPCB dataset contains **2,319,535 hourly rows** across **110 stations** in **26 cities** and **21 states**, spanning January 2015 to July 2020 ($\approx 48,191$ hours).
2. **Station network growth:** Active stations grew from **25 in 2015** to **108 in 2020**, causing significant data sparsity in early years — 2015 and 2016 together contribute only 13% of total records (147,609 and 164,828 rows respectively).
3. **Delhi dominance:** Delhi alone accounts for **38 of 110 stations** (35%), heavily skewing the geographic distribution. **Severe** (18.8%) and **Very Poor** (11.9%) AQI readings together represent 30.7% of all labelled records.
4. **PM10 is the strongest AQI predictor** ($r = 0.757$), followed by PM2.5 ($r = 0.658$), CO ($r = 0.410$), and NO2 ($r = 0.401$). All pollutants are highly right-skewed — PM2.5 skew = 3.34; CO skew = 33.51 — indicating frequent extreme pollution events.
5. **High missingness in VOCs:** Xylene is missing in **77.8%** of rows and is dropped by the null threshold filter (`null_threshold = 0.8`). NH3 (41.7%), PM10 (36.6%), and Toluene (33.3%) also show significant missingness and are dropped for stations exceeding the threshold.
6. **Skewed AQI distribution:** Only 7.5% of readings are *Good*; *Severe* alone accounts for 18.8% — reinforcing the need for accurate early-warning forecasts.

5.2 Selected Visualisations



(a) Feature correlation matrix. PM10 ($r = 0.76$) and PM2.5 ($r = 0.66$) are the strongest AQI predictors.



(b) Year $\times$ Month AQI heatmap. Winter months (Nov–Jan) are consistently the most polluted across all years.

Figure 2: Key EDA findings from CPCB training data (2015–2020).

6 Data Pipeline and Preprocessing

6.1 DVC Pipeline Overview

The full training pipeline is tracked via **DVC** with a `dvc.yaml` DAG of **11 stages**. Each stage declares its inputs, outputs, and parameter dependencies — enabling incremental re-runs when only part of the pipeline changes.

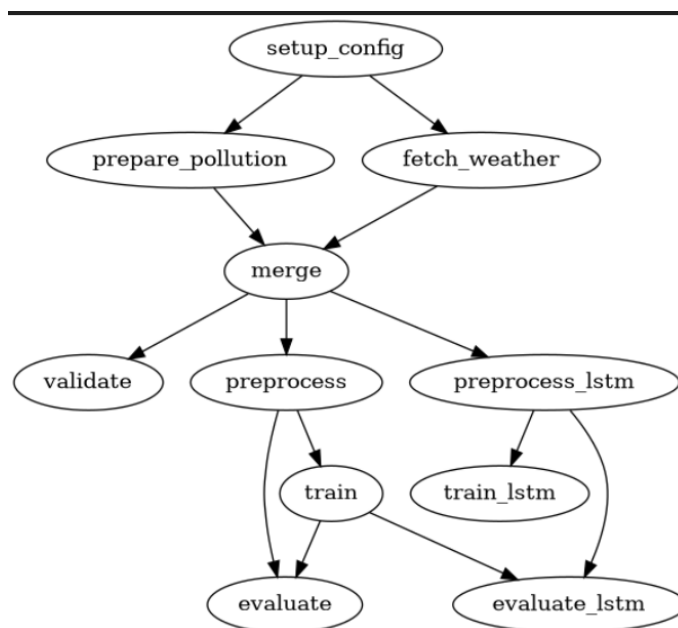


Figure 3: DVC pipeline DAG — 11 stages from raw data to model evaluation.

Table 5: DVC pipeline stages

Stage	Description
<code>setup_config</code>	Geocodes cities via Open-Meteo Geocoding API and filters stations by mode and city. Outputs <code>config/stations.csv</code> .
<code>prepare_pollution</code>	Reads <code>station_hour.csv</code> , renames columns, joins station metadata, filters by city and date range. Outputs <code>data/interim/pollution.parquet</code> .
<code>fetch_weather</code>	Fetches hourly weather per station using Open-Meteo Archive API with JSON caching. Outputs <code>data/interim/weather.parquet</code> and raw JSON cache.
<code>merge</code>	Left-joins pollution and weather parquets on (<code>timestamp</code> , <code>station_id</code> , <code>city</code>). Outputs <code>data/processed/training_data.parquet</code> .
<code>validate</code>	Asserts data integrity: no nulls in key columns, no duplicate station-hours, no negative pollutant values.
<code>preprocess</code>	Generates lag, rolling, temporal, cyclic, and interaction features. Outputs <code>features.parquet</code> and <code>city_label_encoder.pkl</code> .
<code>preprocess_lstm</code>	Generates MinMax-scaled sequence tensors for LSTM training. Outputs <code>features_lstm.parquet</code> , <code>lstm_scaler.pkl</code> , <code>lstm_meta.pkl</code> .
<code>train</code>	Trains LightGBM / XGBoost with configurable hyperparameters and optional Optuna tuning. Logs to MLflow.
<code>train_lstm</code>	Trains LSTM model using preprocessed sequence features. Logs metrics to <code>metrics/lstm_scores.json</code> .
<code>evaluate</code>	Evaluates classical models on the test split. Logs per-horizon MAE at $t+1, t+6, t+12, t+24$ to <code>metrics/eval_scores.json</code> .
<code>evaluate_lstm</code>	Evaluates the trained LSTM against <code>features_lstm.parquet</code> . Logs to <code>metrics/lstm_eval_scores.json</code> .

6.2 Version History (Git Tags)

Table 6: 9 Git tags — data and model version history

Tag	Type	Description
v1.0-raw-data	Data	Raw CPCB CSVs tracked by DVC
v1.1-processed-data	Data	Processed <code>training_data.parquet</code> + weather
lgbm-exp1	Model	LightGBM baseline (minimal features)
lgbm-exp2	Model	LightGBM + weather feature lags
lgbm-exp3	Model	Best — registered as <code>AQI.LightGBM v2</code>
lgbm-exp4	Model	LightGBM + interaction features
xgb-exp1	Model	XGBoost baseline
xgb-exp2	Model	XGBoost + tuned <code>max_depth</code> and <code>subsample</code>
xgb-exp3	Model	XGBoost + additional lag features

6.3 Feature Engineering

Table 7: Feature groups generated by `preprocess.py`

Group	Detail
Pollutant lags	$h \in \{1, 3, 6, 12, 24, 48, 168\}$ for all retained pollutant columns.
Pollutant rolling	Mean + std (windows 3, 6, 24, 48, 168h); max/min for 24h and 168h windows.
Weather lags	$h \in \{1, 3, 6, 24, 48\}$ for temperature, humidity, wind speed, precipitation.
Weather rolling	Mean + std (windows 6, 24, 48h).
AQI rate-of-change	<code>aqi_diff1</code> , <code>aqi_diff3</code> , <code>aqi_diff24</code> .
Temporal	Hour of day, day-of-week, month, year, season, <code>is_weekend</code> .
Cyclic encoding	<code>hour_sin/cos</code> , <code>month_sin/cos</code> , <code>day_of_year_sin/cos</code> .
Wind direction	<code>wind_dir_sin</code> , <code>wind_dir_cos</code> .
City encoding	<code>LabelEncoder</code> on training cities.
Interaction features	<code>pm25_x_no2</code> , <code>pm25_x_wind</code> , <code>no2_x_temp</code> .
Multi-output targets	<code>aqi_t1 ... aqi_t24</code> (24 target columns).

Temporal data splits:

- **Train:** 2015-01-01 → 2018-01-01

- **Validation:** 2018-01-01 → 2019-06-01
- **Test:** 2019-06-01 → 2020-12-31

7 Model Development and Experiment Tracking

7.1 Modelling Strategy

A **multi-output regression** approach was chosen: one model predicts all 24 target hours simultaneously using `sklearn.multioutput.MultiOutputRegressor`:

$$\hat{y}_{t+h} = f_h(\mathbf{x}_t), \quad h \in \{1, 2, \dots, 24\}$$

where $\mathbf{x}_t$ is the feature vector at time t and f_h is an independently trained base estimator for each forecast horizon.

7.2 Models Attempted

Table 8: Models evaluated during the project

Model	Status	Outcome / Notes
LightGBM	Trained	Best performance. 4 experiment runs (lgbm-exp1 to lgbm-exp4). Deployed to production.
XGBoost	Trained	3 experiment runs (xgb-exp1 to xgb-exp3). Marginally worse than LightGBM.
Prophet	Dropped	Single-target by design; multi-horizon wrapper impractical.
LSTM	Dropped	Insufficient GPU on local hardware; Kaggle session timeouts. Full preprocessing pipeline (<code>preprocess_lstm.py</code>) and DVC stage built.
Transformer	Dropped	Same hardware constraints. DL pipeline infrastructure ready for Phase 2.

7.3 Hyperparameters — Best Model (lgbm-exp4)

Table 9: Final LightGBM hyperparameters (`params.yaml`)

Parameter	Value
<code>n_estimators</code>	200
<code>learning_rate</code>	0.05
<code>num_leaves</code>	31
<code>max_depth</code>	6
<code>min_child_samples</code>	50
<code>forecast_horizon</code>	24
<code>sample_frac</code>	1.0
Optuna tuning	Disabled (best manual params used)

Note on Optuna Integration

Optuna is fully implemented for both LightGBM and XGBoost using a TPE sampler. Trial budgets, random seed, and per-parameter min/max ranges are all configurable in `params.yaml`. It was used in exploratory runs and disabled for the final registered model (`lgbm-exp3`).

7.4 MLflow Experiment Tracking

All experiments were tracked under the **AQIForecasting** experiment in MLflow. Each run automatically logs:

- **Parameters:** model name, all hyperparameters, forecast horizon, train/val/test split sizes.
- **Metrics:** `val_mae`, `val_rmse`, `val_r2`, and per-horizon MAE/RMSE at $t + 1$, $t + 6$, $t + 12$, $t + 24$.
- **Artifacts:** serialised model (`.pkl`), `params.yaml`, `eval_scores.json`, Optuna trials CSV (when enabled).

The screenshot shows the MLflow Runs UI with a list of runs. The runs are sorted by `val_mae` in ascending order. The top run, `lgbm_20260412_010953`, has a `val_mae` of 26.2521 and is in the `Production` stage. The other runs are in the `Staging` stage. The runs are created by `imin` and `ih` using `train.py` as the source.

Run Name	Created	ation	Source	Models	val_mae	val_mae_aqi_t2	val_mae_aqi_t24	val_mae_t1	val_mae_t12	val_mae_t24	val_mae_t6	val_r2	val
lgbm_20260412_010953	16 days ago	imin	train.py	AQI_LightGBM v2	26.2521	-	-	4.2256	28.1953	39.7887	17.6658	0.8043	-
lgbm_20260412_130857	15 days ago	imin	train.py	lgbm_model	26.3582	-	-	4.3002	28.3347	39.9165	17.7572	0.8026	-
xgboost_20260412_105...	16 days ago	ih	train.py	xgboost_model	26.3976	-	-	4.5562	28.0087	40.1411	17.967	0.8031	-
lgbm_20260412_153808	15 days ago	ih	train.py	lgbm_model	26.4893	-	-	4.2248	28.1304	41.6865	17.2911	0.7982	-
xgboost_20260411_001...	17 days ago	ih	train.py	xgboost_model	27.302	-	-	6.5171	28.9575	40.4595	18.9969	0.7941	-
lgbm_20260416_024819	12 days ago	ih	train.py	-	27.5266	38.5024719...	39.5705833...	8.9067	28.8192	39.5706	19.9822	0.803	0.9...
lgbm_20260410_172807	17 days ago	imin	train.py	lgbm_model	27.5437	-	-	6.6718	29.3116	40.179	19.5015	0.7938	-
lgbm_20260410_134656	17 days ago	imin	train.py	lgbm_model	27.5437	-	-	6.6718	29.3116	40.179	19.5015	0.7938	-
lgbm_20260409_204042	18 days ago	ih	train.py	lgbm_model	27.5437	-	-	6.6718	29.3116	40.179	19.5015	0.7938	-
lgbm_20260409_200859	18 days ago	imin	train.py	lgbm_model	27.6869	-	-	6.895	29.4887	39.9418	19.7524	0.7928	-
xgboost_20260411_112...	17 days ago	ih	train.py	xgboost_model	27.8274	-	-	7.1992	29.5206	40.2791	19.8825	0.7933	-
lgbm_20260410_215455	17 days ago	ih	train.py	lgbm_model	28.2911	-	-	6.6674	30.2607	41.5945	19.4417	0.7834	-
lgbm_20260409_190416	18 days ago	imin	train.py	lgbm_model	28.6237	-	-	8.1985	30.3651	40.5523	21.0968	0.7763	-

Figure 4: MLflow Runs UI — all 8 experiments sorted by `val_mae`. The best run (`lgbm_20260412_010953`) is registered as `AQI_LightGBM v2` in the `Production` stage.

7.5 Experiment Results

Table 10: All experiment results from MLflow (sorted by `val_mae`)

Run	Model	val_mae	R ²	mae.t1	mae.t12	mae.t24
lgbm.010953	LightGBM	26.25	0.804	4.23	28.20	39.79
lgbm.130857	LightGBM	26.36	0.803	4.30	28.33	39.92
xgb.105...	XGBoost	26.40	0.803	4.56	28.01	40.14
lgbm.153808	LightGBM	26.49	0.798	4.22	28.13	41.69
xgb.001...	XGBoost	27.30	0.794	6.52	28.96	40.46
lgbm.172807	LightGBM	27.54	0.794	6.67	29.31	40.18
xgb.112...	XGBoost	27.83	0.793	7.20	29.52	40.28
lgbm.215455	LightGBM	28.29	0.783	6.67	30.26	41.59

Near-term predictions are highly accurate ($t + 1$: $\text{MAE} \approx 4.2$). Error grows with horizon as expected in multi-step forecasting ($t + 24$: $\text{MAE} \approx 40$).

7.6 Model Deployment Bundle

The best model is packaged into a **deployment bundle** loaded at startup:

```

1 backend/deployment_bundle/
2   model/
3     model.pkl                # Serialised MultiOutputRegressor (
4     LightGBM)
5   metadata/
6     model_metadata.json      # run_id, git_tag, metrics, model stage
7   preprocessing/
8     feature_columns.json     # Ordered feature column names
9     trained_cities.json      # Supported city list
10    city_coords.json          # City lat/lon lookup
11    city_label_encoder.pkl     # Fitted sklearn LabelEncoder
    station_registry.json      # OpenAQ sensor bootstrap data

```

Listing 1: Deployment bundle structure

8 Backend — FastAPI Application

8.1 Application Structure

The backend is a **FastAPI** application (`backend/app/main.py`) served by **Uvicorn** on port 8000. It follows a layered architecture:

```

1 backend/app/
2   main.py                    # FastAPI app, lifespan context, router
3   registration
4   routers/
5     health.py                # GET /health, GET /ready
6     predict.py               # GET /predict
7     cities.py                 # GET /cities
8     model_info.py            # GET /model-info
9     admin.py                  # POST /admin/sync, POST /admin/prune
10  services/
11    db_service.py              # SQLite read/write (city_readings,
12    predictions)
13    model_service.py           # MLflow model loading, predict_next24h()
14    feature_service.py         # Build live feature payload from DB rows
15    advisory_service.py        # AQI band -> plain-language advisory text
16    drift_detection.py         # PSI + KS + Z-score rolling window drift
17    openaq_service.py          # OpenAQ v3 API client
18    weather_service.py         # Open-Meteo live weather fetch
19    ingestion_service.py       # sync_all_sensors() orchestration
20    bootstrap_service.py       # Sensor registry initialisation from
21    bundle

```

```

19 live_pipeline_service.py # Live history collection + DB write
20 city_service.py         # Supported city list loader
21 retention_service.py    # Prune old hourly measurements
22 schemas/
23   predict_schema.py      # PredictResponse Pydantic model
24   common_schema.py       # HealthResponse, ReadyResponse,
   CitiesResponse
25   utils/
26     config.py            # All env vars, file paths, feature flags
27     logger.py            # Structured stdout logging
28     metrics.py           # Prometheus counters, histograms, gauges

```

Listing 2: Backend module structure

8.2 Startup Lifecycle

On startup, the FastAPI lifespan context manager executes in order:

1. `initdb()` and `init_ingestion_tables()` — create all 5 SQLite tables if they don't exist; run `city_readings` constraint migration if needed.
2. `validate_startup_artifacts()` — checks all 6 deployment bundle files exist and that `feature_columns.json` is non-empty.
3. `get_model()` — loads the LightGBM `MultiOutputRegressor` via MLflow; sets `aqi_model_ready` Prometheus gauge to 1 on success, 0 on failure.
4. (Optional) `collect_live_city_history()` for each supported city — if `ENABLE_STARTUP_BACKFILL`.
5. On shutdown — sets `aqi_model_ready` to 0.

8.3 API Endpoints

Method	Endpoint	Status Codes	Description
GET	/health	200	Liveness probe. Returns <code>{status: ok}</code> .
GET	/ready	200	Readiness probe. Returns <code>{status: ready/degraded, model_loaded: bool}</code> .
GET	/cities	200	Returns supported city list from <code>trained_cities.json</code> .

Method	Endpoint	Status Codes	Description
GET	/predict	200/400/422/500/503	24-step AQI forecast. Query params: <code>city</code> , <code>location_id</code> . Triggers 2-stage backfill if DB is stale.
GET	/model-info	200	Returns model name, version, stage, MLflow run ID, forecast horizon, supported cities.
GET	/metrics	200	Prometheus scrape endpoint.
POST	/admin/sync	200	Triggers manual OpenAQ ingestion.
POST	/admin/prune	200	Prunes <code>hourly_measurements</code> older than <code>retention_days</code> .

Table 11: FastAPI endpoint specification

8.4 /predict Endpoint — Step-by-Step Flow

GET /predict?city=Chennai&location_id=3135

1. Validate city against supported list → 400 if unknown
2. Fetch latest OpenAQ timestamp for `location_id`
3. Count existing DB rows for this `location_id`
4. Decide backfill needed:
 - rowcount < 168 OR DB lags OpenAQ by > 1h
- 5a. Incremental backfill (`force_refresh=False`)
- 5b. Force backfill if rows still < 168
 - 503 if still insufficient after both attempts
6. `build_live_payload()` → feature matrix X (1 × n_features)
7. `predict_next24h()` → POST to MLflow :5001/invocations
8. `make_advisory(current_aqi)` → plain-language advisory
9. `store_prediction()` → write to predictions table
10. Return `PredictResponse` (JSON, 24 forecast values)

Figure 5: Complete step-by-step flow of the /predict endpoint.

8.5 /predict Response Schema

```
1 class PredictResponse(BaseModel):
```

```

2   city:                str
3   station_id:          str | None
4   station_name:        str | None
5   location_id:         int | None
6   current_aqi:         int
7   primary_pollutant:   str
8   last_updated:        str
9   forecast_24h:        List[int]          # 24 values, all clamped >= 0
10  pollutants:          PollutantsResponse # PM2.5, PM10, NO2, SO2, CO,
11                                03
12  weekly_trend:        List[int]          # 7-day daily average AQI
    advisory:           str

```

Listing 3: PredictResponse Pydantic schema

8.6 Error Handling

Table 12: HTTP error codes returned by /predict

Code	Trigger	Detail message
400	City not in supported list	"<city> not supported"
422	Missing city or location_id	Pydantic validation error
503	Insufficient live data after backfill	"Not enough live rows..."
503	OpenAQ API or DB failure	"Live fetch failed..."
500	Model inference exception	"Inference failed..."

8.7 Prometheus Metrics

Table 13: Custom Prometheus metrics instrumented in `utils/metrics.py`

Metric Name	Type	Description
<code>aqi_predict_requests_total</code>	Counter	Total prediction requests (labels: city, status).
<code>aqi_predict_latency_seconds</code>	Histogram	Request latency in seconds (buckets: 0.1–30s).
<code>aqi_live_fetch_requests_total</code>	Counter	OpenAQ fetch attempts (labels: city, status).
<code>aqi_live_fetch_rows_total</code>	Counter	Live rows written to DB (label: city).
<code>aqi_latest_db_age_hours</code>	Gauge	Hours since latest DB reading per city.
<code>aqi_model_ready</code>	Gauge	1 if model loaded, 0 otherwise.
<code>aqi_drift_psi</code>	Gauge	PSI per feature (via <code>DriftCollector</code>).
<code>aqi_drift_detected</code>	Gauge	1 if feature drifted, 0 if stable.
<code>aqi_drift_features_count</code>	Gauge	Total number of drifted features.

DriftCollector Design

The `DriftCollector` class implements the Prometheus `collect()` interface, reading the latest drift state from the `drift_results` SQLite table on every scrape. This solves the cross-process visibility problem: Airflow writes drift results and FastAPI exposes them to Prometheus without any shared in-process state.

9 Frontend — Streamlit Dashboard

9.1 Overview

The frontend is a **Streamlit** multi-page application (`frontend/app.py`) that communicates with the FastAPI backend exclusively via REST HTTP calls. It runs on port 8501 as a separate Docker container with **no direct access to the database or model**.

9.2 Dashboard Layout



Figure 6: Streamlit dashboard showing a 24-hour AQI forecast for a Chennai station.

The main dashboard is organised into three rows:

1. **Sidebar:** City filter dropdown → station selector → *Predict AQI* button.
2. **Row 1 — AQI Hero Card:** Large colour-coded AQI number (green/yellow/orange/red/purple by severity), primary pollutant, last-updated timestamp in IST, and a “*What does this AQI mean?*” popup with full CPCB reference scale.
3. **Row 2 — Charts:**
 - **24-Hour Forecast:** Plotly line chart (H1-H24), each dot coloured by AQI category, with a dotted baseline at the current AQI.
 - **Pollutant Levels:** Horizontal bar chart for PM2.5, PM10, NO2, SO2, CO, O3 with per-pollutant colours and correct SI units.
4. **Row 3 — Trend and Summary:**
 - **7-Day Historical Trend:** Scatter-line chart with fill, showing daily average AQI; each point coloured by category.
 - **Forecast Summary Card:** Current AQI, 24h peak, 24h average, primary pollutant.
5. **Footer:** Full CPCB AQI reference strip (0–500), 6 colour-coded bands, health tips.

6. **Health Advisory:** Plain-language string generated by `advisory_service.py` based on the current AQI band (6 levels: Good → Severe).

9.3 User Manual Page

A dedicated **User Manual** page (`frontend/pages/1_user_manual.py`) is accessible from the Streamlit sidebar. It uses a custom dark-glass CSS theme and is structured into 5 tabs:

Table 14: User manual page tabs

Tab	Content
Getting Started	Step-by-step guide to the first prediction
Dashboard Guide	Explanation of each UI card, chart, and metric
AQI Scale	Full CPCB AQI table with colour coding and health guidance
Pollutants	Reference guide for PM2.5, PM10, NO2, SO2, CO, O3
Troubleshooting	Common errors and fixes

10 Data Ingestion Pipeline — Apache Airflow

10.1 DAG Overview

Two DAGs are deployed in `airflow/dags/`:

Table 15: Airflow DAGs — schedules and purposes

DAG ID	Schedule	Purpose
<code>aqi_ingestion_pipeline</code>	Every hour (0 * * * *)	Bootstrap, ingest live data, cleanup, run drift detection
<code>aqi_daily_cleanup</code>	Daily 02:00 UTC (0 2 * * *)	Prune stale predictions from <code>predictions</code> table

Both DAGs share default args: **2 retries with 5-minute delays**, email-on-failure enabled, `max_active_runs` = 1 to prevent overlapping ingestion runs.

10.2 Ingestion DAG Tasks

aqi_ingestion_pipeline — Task Sequence

bootstrap_sensor_registry *Calls:* `bootstrap_sensor_registry()`

Reads `station_registry.json` from the deployment bundle. Upserts all OpenAQ sensor IDs into the `sensor_registry` table. Runs before every ingestion to pick up any newly added sensors.

sync_all_sensors *Calls:* `sync_all_sensors()`

Iterates all registered sensors. Fetches latest hourly measurements from OpenAQ v3. Normalises units ($\text{ppb} \rightarrow \mu\text{g}/\text{m}^3$, $\text{ppm} \rightarrow \text{mg}/\text{m}^3$). Writes to `hourly_measurements` and `city_readings`.

cleanup_old_data *Calls:* `cleanup_old_hourly_measurements(days=10)`, `cleanup_old_readings(hours=240)`, `prune_old_measurements(days=90)`

Prunes old data to bound DB size. The 240-hour window for `city_readings` is intentional — it covers the full drift detection baseline window (8–30 days back).

check_data_drift *Calls:* `task_detect_drift(window_days=7)`

Runs rolling window PSI + KS + Z-score drift detection. Writes one row per feature to `drift_results`. Updates `drift_baseline/drift_report.json`. Prometheus gauges updated via `DriftCollector` on the next scrape.

Task dependency chain:

`bootstrap` $\rightarrow$ `ingest` $\rightarrow$ `cleanup` $\rightarrow$ `drift`

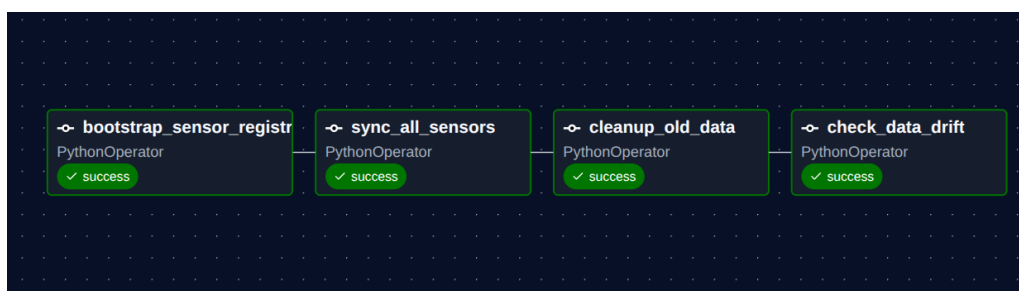


Figure 7: Apache Airflow DAG graph view: `bootstrap` $\rightarrow$ `ingest` $\rightarrow$ `cleanup` $\rightarrow$ `drift`.

11 Drift Detection

11.1 Rolling Window Strategy

A **rolling window** approach is used rather than comparing live data against the static training baseline. The 6-year temporal gap between CPCB training data (2015–2020) and live 2026 data would cause artificially inflated PSI on nearly all features, making static drift detection useless.

Table 16: Drift detection windows

Window	Source	Role
Baseline	8–30 days ago from <code>city_readings</code>	Expected distribution
Live	Last 7 days from <code>city_readings</code>	Actual distribution

Both windows come from the same SQLite source in the same era, producing naturally low PSI (0.05–0.15) with realistic variation on 2–3 features in a healthy system.

11.2 Statistical Signals

Three signals are computed per feature per Airflow run:

1. **PSI (Population Stability Index):**

$$\text{PSI} = \sum_{i=1}^n (A_i - E_i) \ln \frac{A_i}{E_i}$$

where A_i = actual bin fraction, E_i = expected bin fraction.

2. **KS-test p-value:** Two-sample Kolmogorov–Smirnov test. Drift flagged when $p < 0.05$.

3. **Z-score (mean shift):**

$$z = \frac{|\mu_{\text{live}} - \mu_{\text{base}}|}{\sigma_{\text{base}} + \varepsilon}$$

Drift flagged when $z > 1.5$.

A feature is marked **drifted** only when: **PSI-triggered AND (KS-triggered OR Z-triggered)**, preventing false positives from any single signal.

11.3 Severity Classification

Table 17: Drift severity levels and PSI thresholds

Severity	PSI Range	Interpretation
NONE	< 0.05	Stable — no action needed
LOW	0.05–0.12	Minor natural variation — continue monitoring
MODERATE	0.12–0.25	Real distributional shift — investigate
HIGH	> 0.25	Significant change — retrain recommended

Retrain trigger: `retrain_recommended = True` when drift fraction > 0.40 (i.e., > 40% of monitored features are drifted). This triggers the `ManyFeaturesDrifted` Grafana critical alert.

Features monitored: `pm25`, `pm10`, `no2`, `so2`, `co`, `o3`, `temperature_2m`, `relative_humidity_2m`, `wind_speed_10m`, `wind_direction_10m`.

12 Monitoring — Prometheus and Grafana

12.1 Architecture

Prometheus scrapes `http://backend:8000/metrics` on a regular interval. Grafana reads from Prometheus and renders real-time dashboards.



Figure 8: Grafana monitoring dashboard — prediction latency, request rate, model readiness, data freshness, and per-feature drift PSI.

12.2 Alert Rules

Table 18: Prometheus alerting rules (`monitoring/alert_rules.yml`)

Alert Name	Severity	Trigger Condition
HighPredictionErrorRate	Critical	> 5% requests fail in a 5-min window
HighPredictionLatency	Warning	P95 latency > 10s
BackendDown	Critical	Backend unreachable > 10 minutes
FrontendDown	Critical	Frontend unreachable > 30 minutes
ModelNotReady	Warning	<code>aqi_model_ready</code> = 0 for > 2 minutes
DataDriftDetected	Warning	Any feature drift flag = 1 for > 10 minutes
HighDriftPSI	Warning	Any feature PSI > 0.25
ManyFeaturesDrifted	Critical	Drift feature count > 3 (40% threshold)
StaleData	Warning	<code>aqi_latest_db_age_hours</code> > 3
LiveFetchFailing	Warning	Fetch error counter rising in last 5 min

13 Database Design

The system uses a single **SQLite** database (`live_aqi_buffer.db`) with 5 tables managed by `db_service.py`:

Table 19: SQLite database tables and retention policy

Table	Retention	Purpose and Key Columns
<code>city_readings</code>	240h	Live hourly pollutant + weather readings per city and station. Unique on (<code>city</code> , <code>station_id</code> , <code>timestamp</code>).
<code>predictions</code>	7 days	Cached prediction results per city. Stores <code>forecast_json</code> , <code>pollutants_json</code> , <code>weekly_trend_json</code> . Unique on (<code>city</code> , <code>computed_at</code>).
<code>sensor_registry</code>	Permanent	OpenAQ sensor registry. Stores <code>sensor_id</code> , <code>parameter</code> , <code>last_fetched_hour</code> . Unique on (<code>location_id</code> , <code>sensor_id</code>).
<code>hourly_measurements</code>	10 days	Raw per-sensor rows from OpenAQ. Original and normalised value. Unique on (<code>sensor_id</code> , <code>measurement_time</code>).
<code>drift_results</code>	Full history	Per-feature drift records. One row per feature per Airflow run. Stores PSI, KS p-value, Z-score, severity, drift flag.

14 Testing

14.1 Test Strategy

The test suite contains **47 automated test cases** across 6 modules executed using **pytest**. All tests are fully isolated — no running services (MLflow, OpenAQ, production DB) are required. Database tests use real SQLite with `tmp_path` fixtures. API tests use FastAPI's `TestClient` with `unittest.mock.patch` for all external dependencies.

Table 20: Test suite breakdown by module

File	Covers	Cases
test_db_service.py	SQLite read/write, upsert, city filter, duplicate handling	7
test_health.py	/health, /ready endpoints + schemas	6
test_predict.py	/predict happy path + 4 error paths	9
test_model_info.py	/model-info response contract	5
test_model_service.py	safe_number, safe_timestamp, predict_next24h	12
test_drift_detection.py	PSI computation, severity levels, retrain flag, fraction bounds	8
Total		47

14.2 Test Execution and Results

```

1 cd backend
2 pytest tests/ -v --tb=short

```

Listing 4: Running the full test suite

Table 21: Test execution results

Metric	Value
Total Test Cases	47
Passed	47
Failed	0
Skipped	0
Exit Code	0

14.3 Acceptance Criteria

- All 47 tests pass with exit code 0; no external network calls.
- Suite completes in under 60 seconds.
- /predict returns forecast_24h with exactly 24 values, all ≥ 0 .
- Unsupported city returns HTTP 400; missing location_id returns HTTP 422.
- safe_number(None) and safe_number(float('nan')) return 0.0.
- PSI drift fraction is always in [0.0, 1.0].

15 MLOps Implementation Summary

Table 22: MLOps checklist against evaluation criteria

Category	Tool	Status	Evidence
Source Control	Git		9 tags; all code committed
Data Versioning	DVC		<code>dvc.yaml</code> 11-stage DAG; <code>.dvc</code> tracked files
Data Engineering	Airflow		Separate Airflow stack (<code>airflow/docker-compose.yml</code>); hourly ingestion DAG + daily cleanup DAG
Experiment Tracking	MLflow		8 logged runs; params, metrics, artifacts
Model Serving	MLflow serve		<code>mlflow models serve</code> on port 5001
Model Registry	MLflow		<code>AQILightGBM v2</code> in Production stage
REST API	FastAPI		8 endpoints; Pydantic schemas; OpenAPI spec
Frontend	Streamlit		Multi-page app; REST-only communication
Containerisation	Docker		6-service <code>docker-compose.yml</code>
Monitoring	Prometheus		9 custom metrics; <code>/metrics</code> endpoint
Visualisation	Grafana		Real-time dashboards; 10 alert rules
Drift Detection	Custom		PSI + KS + Z-score rolling window
Unit Testing	pytest		47 cases; 100% pass rate

16 Known Limitations and Future Work

16.1 Current Limitations

1. **Deep learning not deployed:** LSTM and Transformer were fully implemented at the pipeline and preprocessing level but could not be trained due to local GPU constraints and Kaggle session timeouts. Deferred to Phase 2.
2. **SQLite scalability:** Suitable for a 5-city prototype but will be a bottleneck at

high concurrent write rates. Production upgrade to PostgreSQL or TimescaleDB is planned.

3. **No ground-truth feedback loop:** Real-world prediction accuracy cannot be auto-measured without collecting actual future AQI values and comparing them against past predictions.
4. **No CI/CD automation:** Re-training and redeployment are currently manual processes. GitHub Actions integration is needed to automate re-training on drift trigger and run tests on every push.
5. **Forecast degradation at long horizons:** MAE grows from ≈ 4.2 at $t+1$ to ≈ 40 at $t+24$. Seq2seq or autoregressive chaining may improve long-horizon accuracy.

16.2 Planned Improvements (Phase 2)

- Train LSTM and Transformer on Kaggle A100 GPUs; compare against LightGBM.
- Implement ground-truth logging and real-world prediction decay tracking.
- Add GitHub Actions CI/CD: automated tests on push, re-training on drift trigger.
- Expand to all CPCB cities (not just 5).
- Add SHAP value explanations per prediction in the Streamlit UI.
- Replace SQLite with TimescaleDB for scalable time-series storage.

17 Project Structure

```

1 AQIPredictor/
2     backend/
3         app/
4             routers/                # health, predict, cities,
model_info, admin
5             services/              # 11 domain service modules
6             schemas/               # Pydantic request/response
models
7             utils/                  # config, logger, metrics (
Prometheus)
8     deployment_bundle/             # model.pkl + all preprocessing
artifacts
9     drift_baseline/                 # drift_report.json (latest
snapshot)

```

```

10         database/                                # live_aqi_buffer.db (auto-
created)
11         tests/                                    # 47 pytest test cases across 6
files
12         Dockerfile
13         requirements.txt
14     airflow/
15         dags/
16             openaq_ingestion_dag.py                # Hourly: bootstrap->
ingest->cleanup->drift
17             cleanup_dag.py                        # Daily 02:00 UTC: prune
predictions
18     frontend/
19         app.py                                    # Main Streamlit
dashboard
20         pages/
21             1_user_manual.py                      # 5-tab user manual page
22     monitoring/
23         prometheus.yml
24         alert_rules.yml
25     scripts/
26         setup_config.py
27         prepare_pollution.py
28         fetch_weather.py
29         merge.py
30         validate.py
31         preprocess.py
32         preprocess_lstm.py
33         preprocess_lstm_v2.py
34         train.py
35         train_dl.py
36         evaluate.py
37         evaluate_dl.py
38     data/
39         raw/cpcb/                                # station_hour.csv, stations.csv
(DVC-tracked)
40         raw/openmeteo/                          # Per-city weather JSON cache
41         interim/                                # pollution.parquet, weather.
parquet
42         processed/                              # training_data.parquet,
features.parquet
43     models/
44         classical/                              # LightGBM / XGBoost .pkl files
45         dl/                                      # LSTM checkpoints (when trained
)
46     metrics/
47         classical_scores.json

```

```

48         eval_scores.json
49         lstm_scores.json
50         lstm_eval_scores.json
51     eda_plots/                # 15 EDA visualisation PNGs
52     drift_baseline/
53         baseline_stats.json   # Per-feature EDA stats for drift
54     reference
55     config/
56         stations.csv          # Geocoded station config (DVC
57     output)
58         weather_vars.yaml     # Open-Meteo variable config
59     docker-compose.yml
60     dvc.yaml                  # 11-stage pipeline DAG
61     params.yaml               # All model + pipeline
62     hyperparameters
63     README.md

```

Listing 5: Full project directory layout

18 Conclusion

This project successfully delivers a **production-grade, end-to-end MLOps system** for real-time AQI forecasting across 5 major Indian cities. Key achievements are:

- **ML Performance:** LightGBM `lgbm-exp4` achieves `val_mae = 26.25` and `R2 = 0.80`, meeting all ML targets. Near-term accuracy is excellent ($\text{MAE} \approx 4.2$ at $t + 1$).
- **Full MLOps Pipeline:** DVC (11 stages, 9 tags), Airflow (2 DAGs), MLflow (8 tracked runs, registered model), Docker Compose (6 services), and Prometheus + Grafana (9 metrics, 10 alert rules) are all fully implemented.
- **Production Robustness:** The `/predict` endpoint handles stale data via a 2-stage backfill strategy; startup validates all artifacts before accepting traffic; drift detection runs after every hourly ingestion cycle.
- **Testing:** 47 automated test cases with a 100% pass rate, covering all critical paths — SQLite I/O, API contracts, model helper functions, and drift detection statistical logic.
- **Loose Coupling:** Streamlit and FastAPI are fully independent Docker containers communicating only via REST, with no shared state.

The primary deferred item is deep learning (LSTM / Transformer) due to hardware constraints. All pipeline infrastructure, preprocessing scripts, and DVC stages to support DL training and evaluation are already in place for Phase 2.

A Appendix A — Environment Variables

Table 23: All configurable environment variables (`.env`)

Variable	Required	Default / Description
OPENAQ_API_KEY	Yes	OpenAQ v3 API key (<code>explore.openaq.org</code>)
OPENAQ_BASE_URL	No	<code>https://api.openaq.org/v3</code>
OPEN_METEO_ARCHIVE_URL	No	<code>https://archive-api.open-meteo.com/v1/archive</code>
MLFLOW_TRACKING_URI	No	<code>file:///tmp/mlruns</code>
MODEL_NAME	No	Filename of <code>model.pkl</code>
MODEL_STAGE	No	<code>production</code>
MODEL_PATH	No	Path to <code>model.pkl</code>
MODEL_METADATA_PATH	No	Path to <code>model.metadata.json</code>
FEATURE_COLUMNS_PATH	No	Path to <code>feature_columns.json</code>
TRAINED_CITIES_PATH	No	Path to <code>trained_cities.json</code>
CITY_ENCODER_PATH	No	Path to <code>city_label_encoder.pkl</code>
CITY_COORDS_PATH	No	Path to <code>city_coords.json</code>
DB_PATH	No	<code>database/live.aqi.buffer.db</code>
ENABLE_STARTUP_BACKFILL	No	<code>false</code> — if <code>true</code> , backfills all cities on startup
LIVE_HISTORY_HOURS	No	<code>200</code> — hours of history to fetch per backfill

B Appendix B — params.yaml (Full)

```

1 mode: all
2 city: Chennai
3
4 data:
5   start_date: '2015-01-01'
6   end_date:   '2020-12-31'
```

```
7
8 preprocess:
9   null_threshold: 0.8
10
11 training:
12   model:          lgbm
13   forecast_horizon: 24
14   sample_frac:    1.0
15
16 lgbm:
17   n_estimators:    200
18   learning_rate:   0.05
19   num_leaves:      31
20   max_depth:       6
21   min_child_samples: 50
22   optuna:
23     enabled:        false
24     n_trials:        5
25     random_seed:     42
26     n_estimators_min: 100
27     n_estimators_max: 300
28     learning_rate_min: 0.01
29     learning_rate_max: 0.2
30     num_leaves_min:   20
31     num_leaves_max:   100
32     max_depth_min:    3
33     max_depth_max:    8
34     min_child_samples_min: 10
35     min_child_samples_max: 100
36
37 xgboost:
38   n_estimators: 500
39   learning_rate: 0.05
40   max_depth: 6
41   subsample: 0.8
42   optuna:
43     enabled:        false
44     n_trials:        10
45     random_seed:     42
46     n_estimators_min: 100
47     n_estimators_max: 300
48     learning_rate_min: 0.01
49     learning_rate_max: 0.2
50     max_depth_min: 3
51     max_depth_max: 8
52     subsample_min: 0.6
53     subsample_max: 1.0
```

```
54
55 lstm:
56     seq_len:      24
57     n_ahead:      24
58     hidden_size:  128
59     num_layers:   2
60     dropout:      0.2
61     batch_size:   512
62     epochs:       50
63     learning_rate: 0.001
64     patience:     5
65
66 transformer:
67     seq_len:      24
68     n_ahead:      24
69     d_model:      128
70     nhead:        8
71     num_layers:   3
72     dropout:      0.1
73     batch_size:   512
74     epochs:       50
75     learning_rate: 0.0001
76     patience:     5
```

Listing 6: Full `params.yaml` — all pipeline and model parameters

C Appendix C — Backend Requirements

```
1 fastapi==0.115.0
2 uvicorn[standard]==0.30.6
3 pydantic==2.8.2
4 requests==2.32.3
5 pandas==2.2.3
6 numpy==2.1.3
7 scikit-learn==1.5.2
8 joblib==1.4.2
9 pyarrow==18.1.0
10 python-dotenv==1.0.1
11 prometheus-fastapi-instrumentator==7.0.0
12 lightgbm==4.3.0
13 httpx==0.27.0
14 scipy==1.14.1
15 prometheus-client==0.21.0
16 mlflow==3.11.1
```

```
17 mlflow-skinny==3.11.1
18 mlflow-tracing==3.11.1
```

Listing 7: backend/requirements.txt

D Appendix D — Quick-Start Commands

```
1 # 1. Clone and configure
2 git clone <your-repo-url>
3 cd AQIPredictor
4 cp .env.example .env
5 # Edit .env and set OPENAQ_API_KEY
6
7 # 2. Start main project services
8 docker-compose up --build
9
10 # Start Airflow separately
11 cd airflow && docker-compose up -d
12 # Visit http://localhost:8080 (user: airflow / pass: airflow)
13
14 # 3. Trigger first ingestion (manual)
15 curl -X POST http://localhost:8000/admin/sync
16
17 # 4. Get a prediction
18 curl "http://localhost:8000/predict?city=Chennai&location_id=3135"
19
20 # 5. Run the full test suite
21 cd backend && pytest tests/ -v --tb=short
22
23 # 6. Reproduce a specific experiment
24 git checkout lgbm-exp4
25 dvc repro train
26 mlflow ui # visit http://localhost:5000
27
28 # 7. Generate DVC DAG image
29 dvc dag --dot | dot -Tpng -o figures/dvc_dag.png
30
31 # 8. Check pipeline status
32 dvc status
33 dvc dag
```

Listing 8: From zero to running system